

武汉大学

《计算机综合项目实践》实验报告

灵犀知径

专业名称：计算机科学与技术

课程名称：计算机综合项目实践

指导教师：刘敏忠

学生学号：

2023302121009、2023302111012、2023302131042

学生姓名：

刘鼎琨、郑慧琳、谢巧

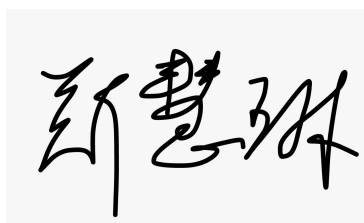
二〇二六年五月

郑重声明

本人呈交的设计报告，是在指导老师的指导下，独立进行实验工作所取得的成果，所有数据、图片资料真实可靠。尽我所知，除文中已经注明引用的内容外，本设计报告不包含他人享有著作权的内容。对本设计报告做出贡献的其他个人和集体，均已在文中以明确的方式标明。本设计报告的知识产权归属于培养单位。

本人签名：_____

日期：2026年5月27日



摘要

随着大语言模型技术的飞速发展，传统静态、缺乏可回溯性的辅助教学模式正面临智能化转型。本文设计并实现了一款基于端云分离架构的多模态智能辅助教学平台——灵犀知径（InsightPath）。系统在表现层采用跨平台框架 Flutter 构建移动端动态渲染引擎，云端依托 Spring Boot 与 Python 构建双后端异构微服务架构，实现业务调度与智能计算的深度解耦。针对垂直领域知识的精准溯源需求，平台提出并实现了基于语义锚点的检索增强生成（RAG）技术，将非结构化切片文本与绝对物理坐标框精准绑定，并持久化至 pgvector 扩展向量库；针对复杂逻辑推演的视觉呈现难题，系统设计了领域特定语言（DSL）交互式动态洞察模块，将云端大模型推理步骤编译为轻量级 JSON 指令流，配合端侧 Canvas 组件内置的插值算法，实现了多端自适应的平滑步进式可视化推演。系统测试结果表明，该平台不仅达成了教学资源全生命周期调度与细粒度权限隔离的目标，更显著提升了代码逻辑诊断与知识点解析的沉浸式交互体验，为人工智能技术赋能现代教育场景提供了创新的工程实践与落地方案。

关键词：智能辅助教学；大语言模型；检索增强生成（RAG）；领域特定语言（DSL）；多模态解析；跨端动态渲染；异构微服务架构

目录

第 1 章 概述	1
1.1 选题	1
1.2 分组及分工	2
第 2 章 系统需求分析	4
2.1 引言	4
2.2 功能需求	4
2.3 非功能需求	8
2.4 技术选型	10
2.5 其他要求	12
第 3 章 系统设计	13
3.1 引言	13
3.2 系统功能设计	14
3.3 系统 UI 设计	17
3.4 系统详细设计	19
3.4.1 基于语义锚点的跨模态 RAG 检索	19
3.4.2 基于 DSL 的生成式动态逻辑渲染	20
3.4.3 基于状态感知的启发式对话引导引擎	23
3.5 数据库设计	25
3.5.1 用户与基础域设计	25
3.5.2 课程与教务流转域设计	26
3.5.3 知识资产与网盘存储域设计	27
3.5.4 索引优化设计	28
第 4 章 系统实现	30
4.1 引言	30
4.2 系统典型界面	30
4.3 测试计划	33
4.3.1 测试策略	33
4.3.2 测试环境	35
4.4 测试报告	35
4.4.1 RAG 检索模块	35

4.4.2 Java 高频网关鉴权与 FSM 路由模块	37
4.4.3 Flutter 端侧 DSL 解释与渲染引擎模块	39
第 5 章 总结	42
5.1 项目总结	42
5.2 贡献评价	42
5.3 未来展望	43

第 1 章 概述

1.1 选题

在计算机科学及理工类学科的高等教育体系中，学生面临知识点高度抽象化与逻辑推导复杂的双重认知挑战。专业核心课程如数据结构、编译原理、计算机网络与操作系统等，包含大量微观层面的状态机转换、内存指针寻址、数据包封装以及线程调度等动态演进过程。这些教学内容不仅要求学生掌握静态的理论定义，更需要建立时序维度的动态逻辑流转模型。然而，现有的教学资源配置通常无法有效支撑这一认知过程的构建。

传统的教学媒介主要依赖于静态的幻灯片课件、纸质教材以及电子文档。这类媒介虽然能够承载详实的理论体系，但在展现复杂的算法演进步骤、数据流转细节以及底层系统调度过程时存在技术短板。这种维度的缺失导致学生在学习过程中产生能够读通文字却无法构建底层逻辑模型的困惑。尽管部分教学团队引入了特定的算法可视化工具，但此类专项工具多采用硬编码形式开发，其展示的场景与输入参数固定，无法随着任课教师个性化教案与自定义资料的更新而自动调整，在实际教学应用中存在扩展性不足的问题。

纵观目前的智能化助学方案，通用型 AI 助手虽然在自然语言交互上具有一定表现，但在专业教育领域面临事实幻觉的技术瓶颈。由于缺乏垂直领域的精确约束机制，通用大模型在解答复杂的理工科推导题时存在输出虚假或逻辑断裂步骤的风险，无法确保回答内容与高校特定教学大纲和教师的授课思路一致。同时，此类工具往往直接给出最终答案，缺乏教育学意义上的步进式、启发式引导策略。此外，传统的学习管理系统与慕课平台维持着无状态的流媒体单向灌输模式，缺乏对学生个性化疑惑点的即时解析能力。

表 1-1 传统教学媒介与检索方案缺陷分析表

评估维度	传统基线方案 (静态 PDF/PPT + 全文检索)	导致的核心缺陷 / 性能瓶颈	本项目目标解法架构定位
时序逻辑表达	仅提供离散状态快照与最终结论	丢失中间态转换逻辑，增加重构计算成本	引入领域特定语言 (DSL) 驱动的过程性动态逻辑渲染
空间坐标	最小寻址单位为页面	无法锚定公式或图表在页面	基于版式分析与 OCR 提取

评估维度	传统基线方案（静态 PDF/PPT + 全文检索）	导致的核心缺陷 / 性能瓶颈	本项目目标解法架构定位
定位	级别（Page-level）	内的二维绝对坐标，上下文易丢失	的二维物理坐标“语义锚点”
交互与检索反馈	单轮开环查询，直接返回静态文本片段	缺乏启发式引导，查准率受限于词库，无思维导图推演	采用跨模态 RAG 架构配合多轮流式对话输出
并发处理开销	依赖教师人工处理 O(N)复杂度的答疑请求	响应延迟极高，教学资源分配的横向扩展性差	部署大模型智能体接管基础推理链条解释

本项目针对计算机及理工类学科教育中静态教学资源难以直观展现动态演进逻辑的现实痛点，研发了灵犀知径智能助学平台。系统通过端云协同模式与动态渲染技术，将教师的隐性教学经验转化为可交互的数字资产，降低学生的认知负荷。本平台通过数字化手段将教师积累的教学经验沉淀为可复用的逻辑知径，将教师从重复性的基础逻辑解释工作中解放出来，使资源匮乏场景下的学习者享受到高质量的逻辑拆解服务，对于提升课程教学质量具备明确的工程应用价值。

1.2 分组及分工

刘鼎琨担任项目的核心中枢，全面负责项目需求分析和整体架构设计。该成员主导整体技术路线，负责系统的云端部署环节，统筹 Docker 容器化交付与算力弹性伸缩技术，确保 Java 业务层与 Python 推理层在生产环境中的平滑隔离与高效通信。在实际开发阶段，刘鼎琨负责构建 Python 算法端的工程基础，配置深度学习环境依赖，封装对接大语言模型的客户端逻辑，并完成 Python 连接 PostgreSQL 数据库底层的开发。在后期联调中，该成员主导解决异构微服务通信选型及 Python 端容器内存溢出等环境问题，并编写正则表达式与数据清洗拦截器对大模型输出的非标准数据进行结构化提取与格式校验。

郑慧琳专项负责前端设计和开发工作。该成员基于 Flutter 跨平台框架进行移动端界面的开发，完成需求规格说明书中用户界面需求和核心交互流程的梳理，负责前端工程目录树与基础依赖配置。在核心功能方面，郑慧琳主导研发 DSL 生成式动态渲染技术，编写专用的解析器与渲染控制器。该成员利用本地 GPU 与底层图形引擎的渲染潜力，通过引入 Provider 架构管理全局状态，并利用 RepaintBoundary 隔离静态图层，解决画布高频重绘导致的性能问题。在界面适

配环节，该成员负责处理软键盘遮挡以及多设备屏幕适配问题，并将抽象逻辑指令流转化为动态交互画布。

谢巧专注于系统的数据底座与算法服务，全面负责后端和数据库的设计和开发。该成员构建以 PostgreSQL 为核心的混合存储基座，设计数据库逻辑结构，并利用 pgvector 扩展插件实现结构化数据与高维语义向量的持久化。在业务逻辑开发方面，谢巧使用 Spring Boot 框架搭建 Java 后端基础架构，编写基于 JWT 的无状态登录鉴权模块、全局异常处理切面及跨域配置。同时，该成员负责实现多态栏目配置与云盘文件管理的核心接口，通过引入 JPA 实体图注解与分离数据库长事务等手段，解决复杂多表查询与高频并发调用中的性能瓶颈，为系统提供数据调度支撑。

第2章 系统需求分析

2.1 引言

本章主要对灵犀知径平台进行系统的需求分析说明。该系统围绕智能启发式教学与交互式动态洞察理念，采用前后端分离与双栈微服务架构设计。在计算机科学及理工类学科的高等教育体系中，传统静态教学媒介在展现算法演进过程与数据流转细节时存在技术短板。本需求分析旨在确立系统通过端云协同模式与动态渲染技术，将教师的教学经验转化为可交互数字资产的各项业务规范与技术边界，以解决静态教学资料难以直观展现动态演进逻辑的问题，降低学生的认知负荷。

本次需求分析涵盖了数据需求、功能需求、性能需求以及接口和其他非功能需求等多个维度。在数据需求方面，系统界定了基础组织字典、权限标识等静态数据的规范，以及账户鉴权流、树状教学编排体系、云盘素材、高维语义向量和画布交互流等动态数据的结构与采集方式。在功能需求层面，依据业务逻辑与用户角色，系统被明确划分为用户认证与权限管理、课程与资源管理、多模态解析与知识构建、智能问答与多模态溯源以及领域特定语言交互式动态洞察五个子模块。

在性能与环境约束方面，需求分析对系统的时间特性与适应性提出了量化要求，规定了基础接口响应延迟、检索召回耗时、首字生成响应时间以及动态渲染帧率的物理阈值。在非功能性维度，本章节明确了系统在跨设备适配、弱网环境断连补偿、容器化水平扩容等方面的限制条件，并设定了传输存储加密、大模型安全防范、接口越权防护以及微服务故障隔离的合规与可靠性标准。上述需求规格的确立为系统后续的概要设计、数据库建模与代码编写提供了基础工程依据。

2.2 功能需求

2.2.1 用户认证与权限管理模块

如表 2.1 所示，用户认证与权限管理模块负责平台用户的身份鉴权、角色路由与安全隔离，支持基于角色的访问控制机制。系统支持学生与教师双角色注册，用户体系与学校实体关联绑定。在认证环节，密码在前端传输时使用 RSA 公钥进行非对称加密，后端采用 BCrypt 算法进行哈希加盐存储。登录完成后，系统签发长效 JSON Web Token 凭证实现状态保持。系统结合 Redis 实现 Token 黑名单机制与双 Token 续签策略，支撑无状态会话管理。用户可修改个人昵称、头像并绑定设备，系统支持多设备登录状态的互斥风控管理。

系统通过全局用户角色标识定义权限边界，角色划分为学生与教师两类。鉴权中间件根据用户角色进行菜单级与接口级路由拦截，限制学生访问教师的课程管理后台。接口访问划分为公开、需登录、需特定角色以及需课程权限四个鉴权级别。在课程维度的安全隔离上，系统定义了课程内成员状态，包括待审核与已正式加入两种状态，用于控制学生对特定课程内部资源的访问与交互权限。

表 2.1 用户认证与权限管理模块功能需求表

功能编号	功能名称	角色	功能描述与约束限制
AUTH-01	双角色注册与认证	访客	支持学生与教师双角色注册。使用加盐哈希算法存储密码。登录成功后系统签发长效 JWT / Session Token，实现状态保持。
AUTH-02	身份信息维护	全员	用户可修改个人昵称、头像，并绑定相关设备。系统支持多设备登录状态的互斥风控管理（踢出异地登录）。
AUTH-03	统一角色路由	系统	鉴权中间件根据用户 Role 自动进行菜单级与接口级路由拦截，确保学生无法越权访问教师的课程管理后台。

2.2.2 课程与资源管理模块

如表 2.2 所示，课程与资源管理模块提供大纲编排、资料上传及设置的全生命周期课程管理能力。该模块涵盖课程从预发布、进行中、已完结到隐藏下架的生命周期流转控制。系统设定了公开、仅限本校与私密三种课程可见性策略，同时组合了开放加入与受限申请两种准入判定逻辑。底层逻辑依据这些策略与状态组合，动态计算学生端在发现页的搜索可见性与课程访问权限，并支持对学生加入请求的业务审批流转。

教师端支持创建树状课程目录，并配置课程内页的栏目体系与排序策略。系统栏目支持多态自定义配置，类型包括用于发布大纲或图文说明的富文本公告栏、

用于管理资料的网盘存储栏，以及用于大模型交互的智能助教对话框。针对特定知识单元，模块允许教师独立配置大语言模型的系统提示词及引导欢迎语，在底层限定模型的回答范围与逻辑边界。

在存储型栏目中，系统实现云盘式文件系统，支持新建文件夹及多层级递归管理。后端基于关系型数据库的自关联表结构设计，采用递归算法与懒加载机制支持多层级文件夹的展开，提供文件的上传、重命名与移动操作。系统通过表单协议上传教学源文件至对象存储服务。在生成文件下载链接时采用临时预签名策略，限制仅具备该课程访问权限的用户在特定时效内获取资源内容，实现防盗链控制。

表 2.2 课程与资源管理模块功能需求表

功能编号	功能名称	角色	功能描述与约束限制
COURSE-01	课程生命周期控制	教师	教师可创建课程，设定其状态（预发布/进行中/完结）及可见性（公开/本校/私密）。支持对学生加入请求的审批流转。
COURSE-02	树状教学大纲编排	教师	支持可视化创建无限层级的树状课程目录（章节-子章节）。每个栏目可定义为富文本页、网盘页或 AI 助教页。
COURSE-03	无限层级网盘管理	教师	在存储型栏目下，支持新建文件夹及多层级递归管理。支持通过 Multipart 协议上传教学源文件（PDF、PPT 等）至云端 OSS。
COURSE-04	定制化 AI 引擎配置	教师	教师可针对特定知识单元，独立配置大语言模型（LLM）的系统提示词（System Prompt）及引导欢迎语，强约束 AI 的辅导边界。

2.2.3 多模态解析与知识构建模块

多模态解析与知识构建模块负责系统对上传教案文件的后台处理流程。系统将文件流的接收与后期的内容解析拆分，采用异步线程处理耗时的解析入库操作，释放了主线程资源。

如表 2.3 所示，在文件解析阶段，模块对非结构化文档进行版式分析与文本内容提取。程序提取出的原文本支持后续的全文检索，并同步提取原文本在课件上的绝对位置坐标框。针对长文本数据，系统引入基于滑动窗口的切片算法，完成对教师上传教案的自动文本切片处理。

完成切片后，系统调用向量模型将文本转化为高维语义向量。提取的绝对位

置坐标框以 JSONB 格式保存，高维语义向量支持高效余弦搜索。这些结构化与非结构化数据最终被持久化到了 PostgreSQL 的 pgvector 扩展库中，完成了教案的语义切片与高维向量化处理。

表 2.3 多模态解析与知识构建模块功能需求表

功能编号	功能名称	角色	功能描述与约束限制
KG-01	文档版式深度解析	系统	异步监听文件上传事件，触发 Python 引擎利用 CV 模型识别并提取 PDF/PPT 中的标题、正文、图像等逻辑块。
KG-02	跨端坐标归一化	系统	计算被提取文本块在原始页面中的绝对物理边界框，并自动转换为 (0~1) 的归一化浮点坐标，作为“语义锚点”入库。
KG-03	高维向量化与索引	系统	串联 Embedding 模型，将文本切片转化为高维语义向量，并写入 PostgreSQL (pgvector) 构建 HNSW 索引。

2.2.4 智能问答与多模态溯源模块

智能问答与多模态溯源模块作为系统核心智能化交互入口，负责为学生提供基于课程上下文的辅助教学服务。该模块涵盖了助教人格化的视觉与引导配置、教师端的系统提示词定制、基于大语言模型的流式对话以及会话历史的闭环管理。模块通过身份隔离机制，在保障教师对助教逻辑具备完全控制权的同时，通过脱敏策略防止核心业务逻辑与敏感提示词向学生端泄露。每一条聊天消息与发起请求的用户实体建立外键联系，并明确归属于特定的课程章节，以便系统精确回溯特定学生在特定知识模块下的认知演进路径。在执行跨模态检索时，系统结合消息实体的上下文，调用存储在向量库中的语义锚点，实现与文件实体中物理坐标的比对，为前端的动态画布渲染及图文对齐功能提供数据中台支撑。具体需求如表 2.4 所示。

表 2.4 智能问答与多模态溯源模块功能需求表

功能编号	功能名称	角色	功能描述与约束限制
QA-01	双路联合语义检索	系统	接收学生提问，并发执行基于向量的语义召回与基于 BM25 的关键词召回，经 Rerank 重排后截取黄金上下文。
QA-02	启发式会话流转	学生	学生在移动端下半屏与 AI 助教进行多轮自然语言对话。系统需结合对话状态追踪器 (DST) 保持逻辑连贯，并严格遵循教师预设的 Prompt 约束，做到“只启发、不代答”。
QA-03	双屏联动与高亮跳转	学生	AI 回复携带引用锚点标签。学生点击后，上半屏文档查看器瞬间执行“翻页 -> 相对坐标反算 -> 像素级黄色遮罩高亮”的平滑图文对齐动效。

2.2.5 DSL 交互式动态洞察模块

领域特定语言交互式动态洞察模块负责将云端推理出的抽象逻辑步骤转化为前端可视画布的控制逻辑。在数据流转链路上，智能计算层的大模型将复杂的推导步骤编译为轻量级领域特定语言（DSL）指令流，表现形式为统一的 JSON 格式。为确保数据在传输过程中的合规性与完整性，Python 算法端在流式输出逻辑中引入正则表达式，对模型生成的 DSL 动画指令进行结构化提取和格式校验，剔除外层包裹的 Markdown 标记或未闭合的括号，确保下发的指令数据完全符合既定协议规范，防止前端强类型解析引发系统崩溃。

表 2.5 DSL 交互式动态洞察模块功能需求表

功能编号	功能名称	角色	功能描述与约束限制
DSL-01	云端意图抽象与下发	系统	大模型推理出算法演进逻辑后，由 Python 状态机将其编译为轻量级 DSL JSON 指令流（包含 INIT, SWAP, UPDATE 等原子动作）下发至移动端。
DSL-02	端侧状态驱动渲染	学生	移动端 Flutter 解析 DSL 报文，不处理复杂像素计算，仅更新微组件的逻辑状态，利用框架隐式动画引擎完成 60FPS 的丝滑视觉重绘。
DSL-03	多维时空交互控制	学生	支持学生在动画演示过程中，通过手势拖拽进度条实现“时间回溯”（重置帧状态）；支持修改画布上的辅助变量（如节点数值），触发云端重新推理并下发新指令。

2.3 非功能需求

2.3.1 性能需求

本系统融合了深度学习推理与高频实时渲染，为保障学生在复杂交互场景下的沉浸式体验，系统在响应延迟、吞吐量及多端适配上确立了严格的性能指标要求。

表 2.6 性能需求表

性能指标维度	目标阈值	详细说明与约束
基础接口响应 (API Latency)	≤ 300 ms	常规业务接口（如获取课程列表、鉴权校验、查询大纲）在 P95 水位下的端到端响应时间不超过 300 毫秒。

双路召回耗时 (RAG Retrieval)	≤ 800 ms	在百万级向量切片规模下，pgvector 密集检索与 BM25 稀疏检索的并发双路召回及 Rerank 重排序总耗时需控制在 800 毫秒以内。
首字响应时间 (TTFT)	≤ 2.0 s	学生提交问题后，系统大模型推理出第一个 Token（首字）并推送到移动端的时间（Time To First Token）不得超过 2 秒，且必须采用 SSE/WebSocket 流式输出。
图文对齐渲染延迟	≤ 100 ms	移动端接收到归一化溯源坐标后，驱动 PDF 渲染引擎进行页面滚动、坐标逆运算及遮罩层重绘的整体视觉延迟需低于 100 毫秒，实现“即点即达”。
DSL 动态渲染 帧率 (FPS)	≥ 60 FPS (单帧 ≤ 16ms)	Flutter 端解析 DSL JSON 指令流并驱动组件状态机重绘时，需保证动画的丝滑过渡，严禁出现主线程阻塞，帧率需稳定在 60 FPS（即每帧渲染耗时低于 16 毫秒）。

2.3.2 硬件接口需求

移动端（Flutter 客户端）在运行过程中，需合法合规地调用底层设备的硬件资源，以保障动态渲染性能与基础业务功能的正常运转。

表 2.7 硬件接口需求表

硬件类型	调用目的与业务场景	性能约束与调用限制
GPU 图形处理器	支撑 Flutter 底层渲染引擎（Skia / Impeller）处理复杂的 Canvas 绘制指令，为 DSL 动态逻辑动画提供硬件级图形加速。	需确保在高频帧插值运算时优先调度 GPU 资源，避免占用过多 CPU 导致设备发热或主线程阻塞。
物理存储介质 (ROM/RAM)	读写设备本地存储，用于持久化缓存已下载的 PDF 教案资料、离线 Markdown 文本及用户的身份 Token。	必须严格遵守 Android/iOS 的沙盒存储规范（Scoped Storage），严禁越权扫描用户非应用目录文件。
触控与手势传感器	采集用户在屏幕上的多点触控（Pinch-to-zoom）与滑动位移数据，驱动前端界面的视窗缩放与动画时间轴拖拽。	要求采样率不低于 60Hz，确保手势识别与画布重绘之间的延迟低于 16ms。
网络适配器 (Wi-Fi/基带)	维持移动端与云端服务器之间的持久化 TCP/WebSocket 连接，接收流式数据包。	需监听底层网络状态切换（如从 Wi-Fi 切至 5G），并在弱网环境下自动触发降级与断线重连重传机制。

2.3.3 内部通信接口需求

表 2.8 内部通信接口需求表

接口分类	通信协议与数据格式	交互场景与说明
BFF 网关路由接口	RESTful	移动端/Web 端与云端网关的数据交换通道。统一由网关进行 JWT 鉴权剥离与限流防刷拦截，随后将流量路由至下游业务集群。
	HTTP/1.1	
	请求/响应格式: JSON	
微服务跨栈 RPC 接口	gRPC / HTTP 内部调用	Java 业务服务与 Python 智能计算服务之间的高速通信隧道。如: Java 端接收到学生提问后，通过此接口将上下文和用户信息透传给 Python 端触发 RAG 检索引擎。
	数据格式: Protobuf/JSON	
	SSE (Server-Sent Events)	
流式数据推送接口	数据格式: Text/Stream	Python 智能层大模型生成回答与 DSL 渲染指令时，利用 SSE 协议向前端发起单向流式推送，实现打字机效果及实时画面渲染。

2.4 技术选型

系统的表现层前端开发采用 Google Flutter 框架与 Dart 语言环境。客户端调用绘图 Canvas 组件执行逻辑画布重绘。前端重构网络请求逻辑，引入服务器发送事件（SSE）监听机制适配流式下发协议。在动态渲染方面，前端协议将物理坐标转化为相对百分比坐标，利用 MediaQuery 组件动态获取设备物理宽高执行换算，实现图形等比缩放。解析引擎在 Canvas 组件内部集成 AnimationController 与 Tween 插值器计算中间过渡帧，更新页面坐标。

系统业务应用层与智能计算层采用 Java 与 Python 异构架构。业务网关层基于 Spring Boot 框架构建，承担用户鉴权、安全路由与 API 网关分发任务。智能计算层运行在 Python 环境中，负责处理文本解析、滑动窗口切片算法执行以及大模型通信逻辑封装。两层微服务之间采用基于容器内部网络的 HTTP/REST 协议进行数据对接。

数据持久层采用结构化数据与非结构化向量的混合存储模型。关系型数据选用 PostgreSQL 数据库，启用 pgvector 扩展插件支持高维语义向量存储与空间余弦相似度计算。持久化框架引入 JPA 实体图注解聚合拉取关联数据，并将网络

操作剥离出事务作用域。

语义锚点 RAG 技术通过异步线程执行版式分析与内容提取，利用切片算法将长文本切分为固定长度文本块，同步提取文本绝对位置坐标框。系统将切片文本转化为高维语义向量，连同坐标框数据持久化至 pgvector 扩展库。答疑时系统结合上下文召回语义锚点组装提示词，实现文本特征与绝对物理坐标绑定。

DSL 动态渲染技术在智能计算层将推导步骤编译为领域特定语言 JSON 指令流。Python 端配置正则表达式拦截器进行数据清洗与格式校验。系统基建层基于容器化技术构建，微服务与数据库通过 Dockerfile 封装，由 docker-compose 文件进行联动部署。系统在 Java 启动脚本中增加 JVM 堆内存上限约束，并在 Python 端改用单线程异步机制，控制运行峰值内存消耗。

表 2.9 技术选型表

技术维度	选用技术	核心实现内容
表现层	Google Flutter & Dart	客户端调用 Canvas 重绘；采用 Provider 架构；配置 SafeArea 区域。
业务应用层	Spring Boot & JDK 17	承担鉴权、路由及网关分发；通过 SecurityFilterChain 配置安全策略；实现全局异常切面处理。
智能计算层	Python & Pyvenv	封装模型通信逻辑；执行版式分析与滑动窗口切片；通过单线程异步处理词向量。
数据持久层	PostgreSQL & pgvector	建立混合存储模型；利用 pgvector 存储语义向量；引入 JPA 实体图注解减少多表查询开销。
语义锚点 RAG	RAG 检索增强生成	异步提取文本及坐标框并向量化存储；答疑时结合上下文召回语义锚点组装提示词。
DSL 动态渲染	领域特定语言 (JSON)	模型编译 JSON 指令流；Python 正则校验；前端通过 SSE 机制接收，利用插值器计算过渡帧。
基建与部署	Docker & docker-compose	通过 Dockerfile 与 docker-compose.yml 容器化部署；对 Java 服务配置 JVM 内存上限约束。

2.5 其他要求

除上述功能、性能与接口需求外，本系统在安全性、可靠性、可维护性及数据合规性方面，具有以下非功能性需求约束。

表 2.10 其它需求表

需求维度	核心需求项	详细说明与技术约束
1. 安全性需求 (Security)	传输与存储加密	系统全链路必须采用 HTTPS/TLS 1.2+ 协议进行加密传输；用户密码等敏感信息必须采用加盐哈希（如 Argon2 或 bcrypt）不可逆算法进行持久化存储。
	大模型安全规范	针对 AI 交互场景，系统必须具备“防 Prompt 逆向注入”机制。后端绝对不可向前端明文暴露教师设定的 system_prompt，且需对学生的异常提问（如诱导越狱指令）进行前置拦截。
	接口越权防护	所有受限业务接口必须经过 JWT 鉴权校验；必须实现基于角色（RBAC）与资源归属者（如：学生只能查看自己加入的课程）的细粒度越权防护（防止横向与纵向越权）。
2. 可靠性需求 (Reliability)	微服务故障隔离	依托 Docker 容器化部署，Java 业务服务与 Python 算法服务需实现物理与逻辑隔离。若 Python 计算节点因算力耗尽宕机，核心业务栈（注册、登录、静态文件查看）需保持可用。
	云端 API 灾备机制	针对第三方大模型 API 的调用，系统需具备“多模型轮询备用”与“超时熔断”机制。当主选推理模型（如深浅推理层）不可用时，能在 3 秒内无缝切换至备用模型，保障教学中断。
3. 可维护性需求 (Maintainability)	DSL 协议版本控制	考虑到渲染指令流后续的持续迭代，云端下发的 DSL JSON 报文必须包含协议版本号（如 "version": "1.0"）。前端需具备向下兼容解析能力，确保旧版本 App 不会因指令升级而崩溃。
	全链路监控与日志	核心业务流与 RAG 检索流需接入标准化的日志采集规范。针对大模型生成的错误/幻觉内容，提供统一的 Trace ID 以支持多轮对话上下文的链路追踪与复盘调试。
4. 合规与隐私 (Compliance)	知识产权与隐私保护	教师上传的教案资料（PDF/PPT 等）涉及知识产权，必须存储于带权限控制的私有 OSS Bucket 中；学生的问答历史记录在脱敏前，严禁被直接用于公共大语言模型的二次微调训练。

第 3 章 系统设计

3.1 引言

本章主要对灵犀知径平台的系统方案进行详细的设计说明。平台采用端云分离与多语言微服务协同的分布式架构设计。系统在逻辑上划分为表现层、业务应用层、智能计算层与数据持久层四个板块，构建起从前端交互感知到后端算法推理的控制流程。该架构通过容器化沙盒机制实现算法迭代与业务逻辑的解耦，为各组件的运行提供工程支撑。

在系统各层级的具体设计中，表现层针对学生端与教师端的不同操作诉求进行差异化设计。移动端采用跨平台框架实现动态绘图引擎，教师端依托 Web 技术构建资料上传与干预策略配置的编排工作台。业务应用层承担用户鉴权、课程进度管理及网关分发任务，智能计算层独立处理文本解析、切片算法与模型通信逻辑。数据持久层建立结构化业务数据与非结构化向量的混合存储基座，支撑跨模态检索的底层调度。

本章的后续内容涵盖系统总体功能模块的层次划分、针对双角色的移动端界面视图导航规划、动态逻辑渲染与多模态溯源的详细执行流程设计，以及数据库表结构与向量实体关联的底层建模。本部分设计工作将需求分析阶段确定的各项业务边界与性能约束，转化为具备可执行性的软件工程物理模型与数据规范，为后续的系统编码与端云联调实现确立开发标准。

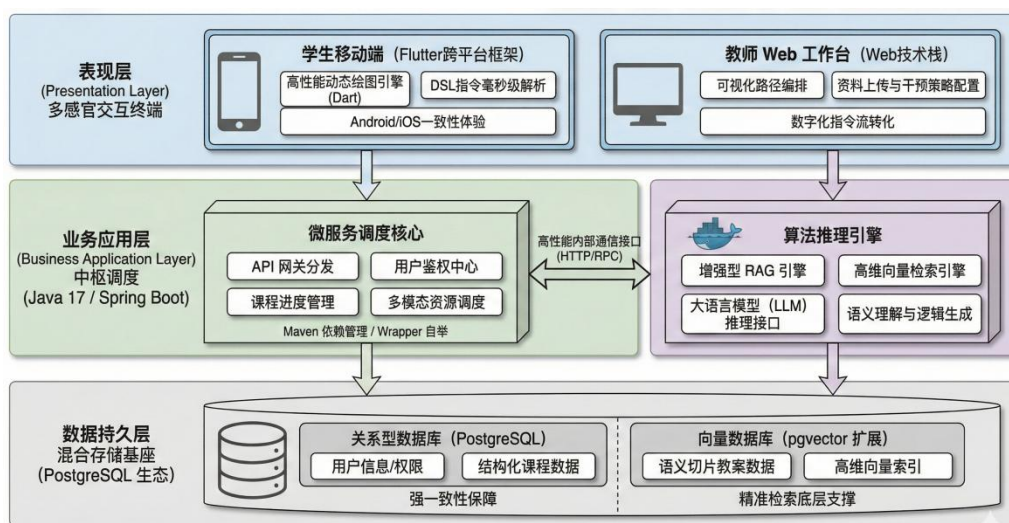


图 3.1 灵犀知径平台总体架构图

3.2 系统功能设计

系统的功能设计采用模块化与解耦思想，依托多语言微服务框架实现端到端的数据流转控制。总体功能架构垂直划分为基础支撑组件、核心业务调度单元与智能计算中枢。系统在逻辑层面采用应用编程接口进行通信，各功能模块通过标准数据传输对象封装底层实体，隔离业务逻辑与数据库表结构，利用轻量级网络协议执行跨容器的状态同步与参数传递。

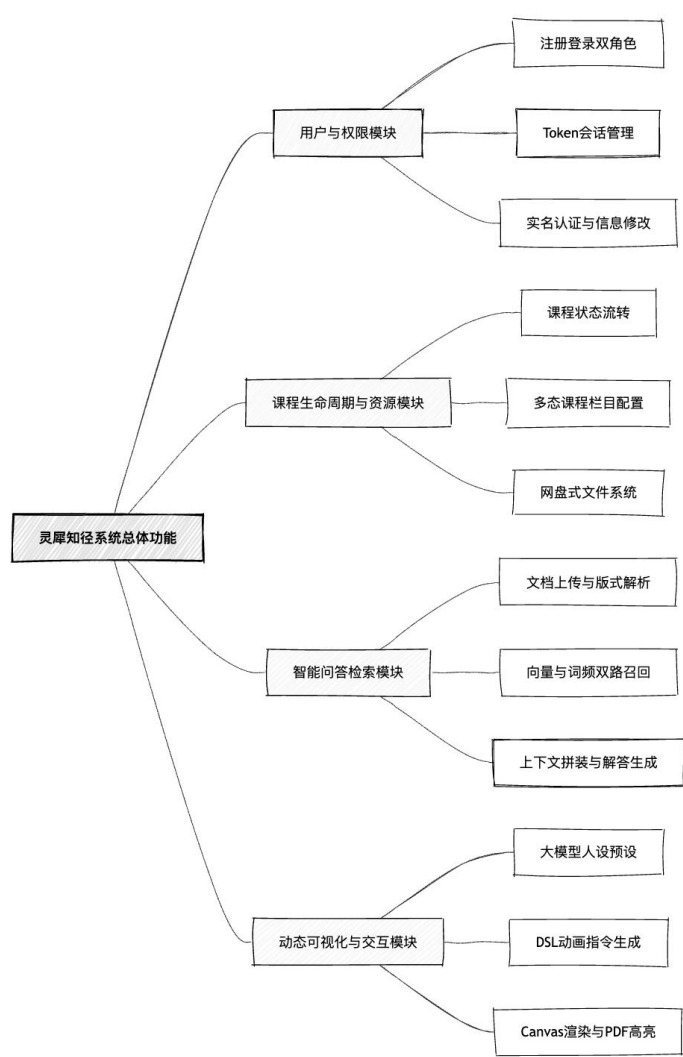


图 3.2 灵犀知径系统功能结构图

3.2.1 用户认证与权限管理模块设计

用户认证与权限管理模块在设计层级主要负责平台用户的访问准入、身份安全校验与角色的路由控制。系统的安全配置基于 Spring Security 框架，系统在安

全管理类中采用基于安全过滤链组件化的配置方式。这一设计方案通过重写安全过滤链，明确放行本地开发及前后端联调过程中出现的跨域预检请求，确保了网络通信链路的连通。为防范网络传输中的凭证泄露，登录阶段的用户密码在前端通过加密处理，后端业务系统接收并解密后，调用算法执行哈希加盐运算，并将计算得出的数据落盘持久化至用户基础表中。

鉴权机制在设计上引入无状态会话规范，系统在用户成功通过登录验证后为其签发凭证，前端后续的业务请求均需在请求头中携带该标识。为了解决该凭证在无状态设计下难以主动注销的缺陷，系统利用缓存中间件构建了凭证黑名单结构，并部署了双凭证续签策略。在针对资源层面的防护上，设计方案在过滤器链路中配置了越权拦截器，系统实时读取请求上下文中解析出的全局用户角色标识以及当前的课程内成员状态常量。拦截器对不符合访问规则的请求执行拦截并抛出异常，由全局异常处理切面统一捕获并转化为标准响应结构体下发，从而实现接口级别与资源层面的隔离防护。

3.2.2 课程与资源管理模块设计

课程与资源管理模块的设计侧重于复杂层级教学数据的调度与生命周期控制。系统在业务逻辑层构建了对应的控制类接口，处理课程创建、成员权限隔离以及教学大纲的树形结构管理。针对课程模型嵌套多层级教学大纲与章节内容的复杂数据结构，数据持久层废弃了实体类直接对外暴露的结构，建立数据传输对象层以物理隔离双向映射引发的序列化死循环。在查询逻辑设计上，持久化框架引入实体图注解配置，通过底层数据库的左外连接操作，在单次结构化查询语言会话中聚合拉取强关联的子节点数据，消除了深层嵌套引发的多表独立查询性能瓶颈，将接口响应延迟控制在物理阈值以内。

资源调度功能的设计涵盖了教案文件的物理转存与访问控制。系统在文件上传接口设计中，通过调整配置文件中的全局请求与文件体积参数上限，以支持大体积源文件的传输。文件读写逻辑采用了解耦设计，系统将文件流的接收与后续的内容解析步骤进行分离。程序主线程负责文件的接收与响应，耗时的内容解析与入库任务则交由异步线程独立处理，从而释放主线程资源并规避接口请求超时限制。在资源访问链路设计中，模块采用临时预签名策略，通过生成携带时效性签名的下载链接，限定资源仅向具备当前课程访问权限的用户开放，实现存储对

象层的越权防范与防盗链控制。

3.2.3 多模态解析与智能问答模块设计

多模态解析与智能问答模块的设计围绕检索增强生成管线与底层数据重构流程展开。在文档预处理阶段，系统将非结构化教案的处理逻辑独立封装于智能计算层中。系统调用版式分析工具，同步提取页面原文本与对应的绝对位置坐标框数据。针对长文本数据，设计方案引入滑动窗口切片算法执行文本段落分割，以控制检索段落的边界与长度。系统随后调用模型将切片文本转化为高维语义向量，并连同以 JSONB 格式封装的物理坐标框数据，一并持久化保存至 PostgreSQL 的 pgvector 扩展库中，完成检索数据的物理结构构建。

在查询召回与模型通信阶段，系统设计了跨模态的上下文检索链路。当接收到自然语言指令后，模块将其转化为高维向量，在向量扩展库中执行空间余弦相似度计算，结合消息实体的上下文定位关联的语义锚点与绝对物理坐标。系统将检索召回的文本与系统提示词进行约束组装，传递给大语言模型进行逻辑推演。在通信控制层，算法端封装了对接模型的客户端代码，优化请求逻辑以接入流式输出，并通过重构底层的网络请求逻辑，引入服务器发送事件监听机制建立流式数据下发通道。在输出链路上，程序内部配置了基于正则表达式的数据清洗拦截器，对模型输出的数据进行结构化提取与格式校验，剔除外层包裹的代码块标记或未闭合字符，确保下发给前端的数据符合既定协议规范。

3.2.4 领域特定语言交互式动态洞察模块设计

领域特定语言交互式动态洞察模块负责将云端的推导步骤转化为前端的视觉控制流。在数据下发链路中，智能计算层将算法演进过程编译为标准格式的 JSON 指令流。为保障前端强类型解析的稳定性，算法端在流式输出逻辑中增加了针对领域特定语言指令的数据清洗功能。系统引入正则表达式，对模型生成的内容进行结构化提取和格式校验，强制剔除外层包裹的 Markdown 代码块标记以及因截断导致的未闭合字符，确保输出的指令序列符合既定协议要求。

在表现层的数据接收与坐标定义上，前端重构了底层的网络请求组件，引入服务器发送事件监听机制来适配后端的流式数据下发协议。为解决跨端设备屏幕尺寸差异导致的图形拉伸与溢出问题，系统在定制动画指令协议时，将物理坐标统一换算为零到一之间的相对比例坐标。前端解释引擎在接收到数据后，利用

MediaQuery 组件动态获取当前持有设备的物理屏幕宽高，通过动态乘法换算将比例值还原为绝对像素坐标，从而完成自适应排版。

在画布图形重绘与界面自适应设计中，前端基于跨平台框架提供的 Canvas 组件执行底层渲染。针对坐标刷新时图形瞬间跳变的视觉缺陷，模块在解释器内部引入了 AnimationController 与 Tween 插值器。每次接收到新坐标指令后，程序并不直接对图形进行硬件赋值，而是以当前位置为起点、目标位置为终点，利用插值算法计算生成中间的过渡帧坐标，配合设备的硬件刷新率逐步更新页面。在处理输入法冲突时，系统动态获取设备底部键盘的高度，将页面外层组件的属性设为静态约束，取消系统默认的硬性压缩行为，并通过手动计算边距维持图形界面的物理完整性。

3.3 系统 UI 设计

3.3.1 界面架构与基础视图

系统终端界面基于跨平台引擎构建，路由策略在物理上划分为鉴权前与鉴权后两套独立栈。鉴权前视图由登录与注册页面构成，采用垂直流式布局接收文本凭据与表单动作。鉴权后视图依据服务端下发的角色标识，重定向至教师工作区或学生工作区。主工作区统一采用底部导航栏完成一级功能模块的物理级切换。个人中心模块提供用户属性更新入口，支持资料文本表单交互与头像媒体资源的选择裁剪上传。

3.3.2 核心业务流转界面

教务流转场景的界面依据角色权限呈现差异化布局。教师端包含独立的课程创建与设置表单界面，允许用户输入基础元数据并调起系统相册选择封面图片。课程详情内页采用顶部水平标签页结构，将大纲节点编排、递归文件目录树与成员审批表单分为三个独立的横向滑动子视图。文件目录视图采用按需交互模式，单击文件夹条目触发网络请求并局部渲染下级元素。学生端界面隐藏了所有的结构变更控件，整体布局向内容消费倾斜，侧重于富文本渲染、测验载入与进度状态追踪。



图 3.3 系统业务流转页面示例图

3.3.3 动画渲染与智能对话视图

交互式动画播放器采用全屏沉浸式布局，剥离了非必要的基础框架控件。核心区域为覆盖界面主体的自定义画板控件，底层叠加高斯模糊滤镜处理，前景色利用绝对坐标定位引擎绘制数据实体与连接边。界面顶部挂载半透明的文本卡片用于渲染步骤描述，底部固定悬浮控制台，封装了重置、后退、暂停与步进四个状态控制触发器。全局助教与局部文档问答采用标准流式对话视图，输入组件固定于屏幕底边，对话记录面板支持垂直手势滚动，通过气泡组件的颜色与左右对齐位置区分终端输入与模型推演输出文本。

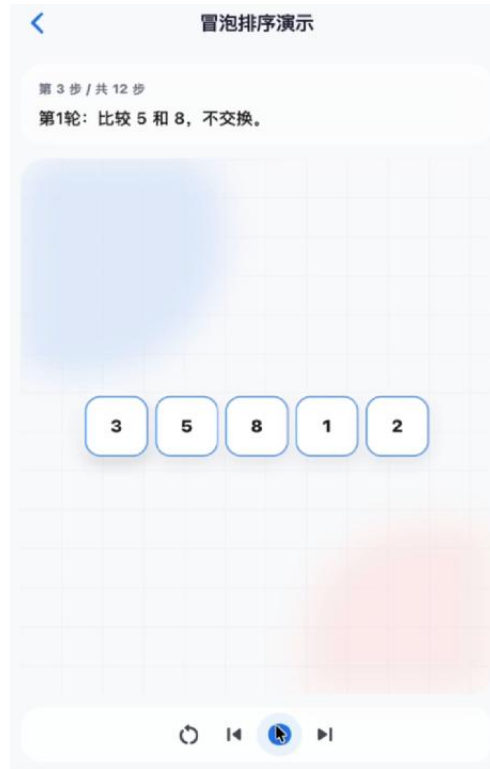


图 3.4 DSL 动画示例图

3.4 系统详细设计

3.4.1 基于语义锚点的跨模态 RAG 检索

本项目在 PostgreSQL 关系型数据库与 pgvector 向量扩展的基础上，设计并实现了基于语义锚点（Semantic Anchors）的跨模态检索增强生成（RAG）技术（总体架构参见图 3.5）。在数据预处理阶段，系统集成 OCR 引擎与版式分析模型对非结构化教案进行解析。提取过程以“语义完整段落”或“独立图表结构”为最小文本基元（Chunk），通过滑动窗口算法将切片粒度动态控制在 256 至 512 个 Token 之间，以规避硬性截断导致的上下文断裂。

系统为切分出的每个文本基元强绑定了一套五维物理属性元数据，具体字段包括：所属文档标识（document_id）、所处物理页码（page_number）、逻辑组件类型（element_type，如 Text、Formula、Table）、以及在当前页面二维坐标系中的绝对边界框坐标（bounding_box: [x_min, y_min, x_max, y_max]）。上述由页码与边界框坐标构成的空间位置约束，在系统中被统称为“语义锚点”。

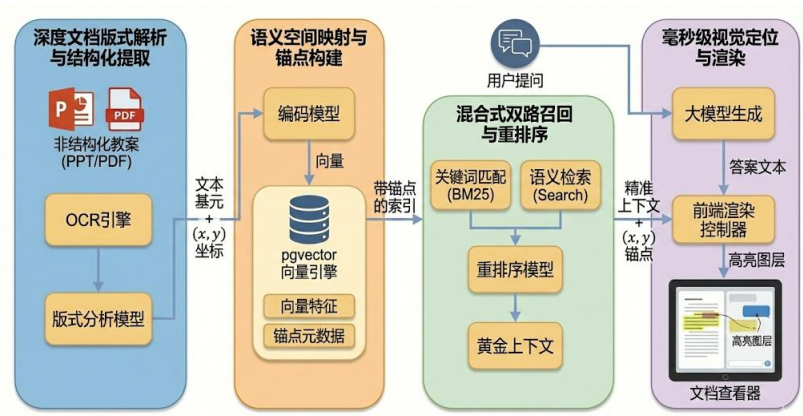


图 3.5 基于语义锚点的跨模态 RAG 增强技术框架图

在特征持久化层面，系统采用多语言编码模型 bge-m3 将离散文本序列稠密化为 1024 维语义特征向量。上述高维张量与其绑定的五维语义锚点元数据，经由异步批处理写入 pgvector 引擎。为满足目标查询吞吐率，底层存储构建了基于分层导航小世界 (HNSW) 算法的近似最近邻 (ANN) 索引，并指定余弦相似度 (Cosine Similarity) 为距离度量标量。该拓扑结构在物理层面上完成了从抽象语义空间至二维视觉坐标系的映射双重绑定。

检索与生成链路被实例化为“双路召回-交叉重排-约束生成”的三段式管道。系统并行执行基于词频-逆文档频率的稀疏匹配 (BM25) 与基于 HNSW 的高维语义检索，由倒数秩融合算法 (RRF) 对两路异构得分集进行无监督权重对齐，并截断输出 $Top - 20$ 候选簇。随后，该候选集被输入至参数量为 278M 的 bge-reranker-base 交叉编码器执行相关性二次评估，最终收敛至 $Top - 5$ 的最优上下文分块。在大语言模型生成阶段，系统通过系统级提示词施加推理边界，强制其输出附带目标 (x, y) 坐标的结构化响应。前端渲染控制器解析该 JSON 负荷后，以 $O(1)$ 的时间复杂度驱动文档视图跳跃至目标物理页码，并在指定坐标区域调用图形 API 绘制高亮图层。此数据闭环从源头阻断了自回归模型的事实性漂移，保障了输出知识的可溯源性。

3.4.2 基于 DSL 的生成式动态逻辑渲染

针对理工科教学中复杂动态过程（如算法指针演变、物理状态转移）的表达需求，本项目设计并实现了一套基于领域特定语言 (Domain-Specific Language, DSL) 的端云协同渲染架构（系统物理拓扑与数据流向详见图 3.6）。该架构废

弃了基于密集像素采样的传统视频推流协议，转而采用低带宽的声明式逻辑指令流驱动端侧状态机。

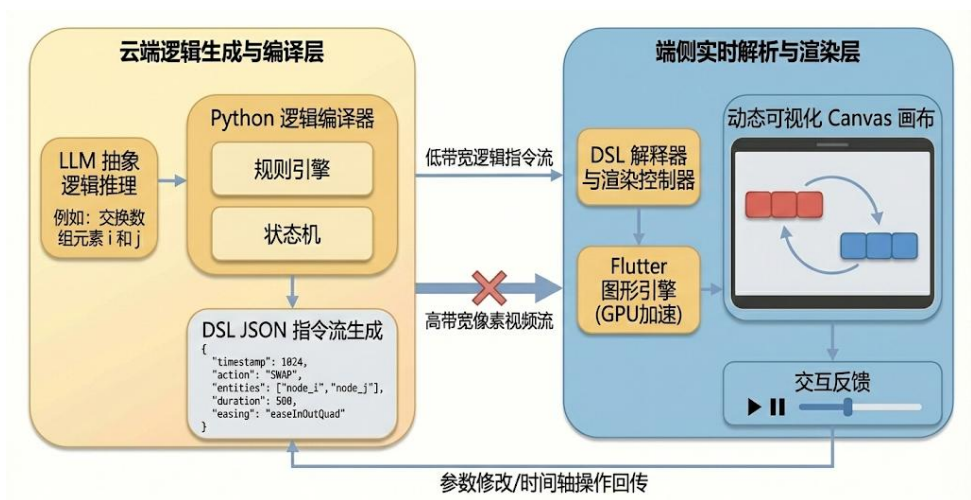


图 3.6 基于 DSL 的交互式动态逻辑生成与渲染技术原理图

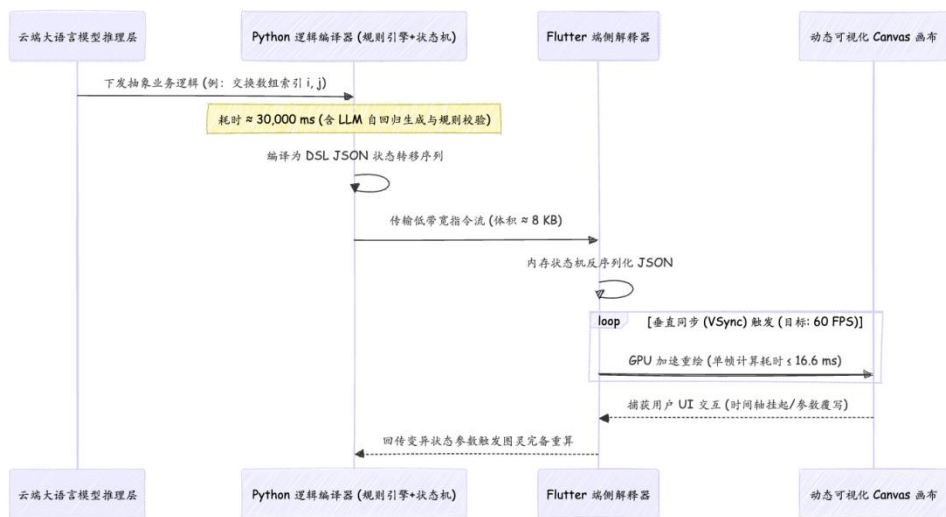


图 3.7 DSL 生成式动态渲染系统交互与数据流转时序图

在云端计算层，系统部署了由规则引擎与有限状态机 (FSM) 组成的 Python 逻辑编译器。当大语言模型 (LLM) 输出抽象的推理结果后，编译器以 $O(|V| + |E|)$ 的时间复杂度遍历当前画布的虚拟 DOM 节点集合 (V) 与逻辑边集合 (E)。结合上下文环境，编译器将自然语言描述的动作转化为结构化的 DSL 序列。

经系统压测统计，包含完整多轮算法推演的云端联合处理延迟（包含 LLM 推理与编译逻辑校验）被限制在约 30,000ms。编译完成的 DSL 指令流以 JSON

格式下发, 平均包含约 200 行键值对, 单次网络传输的有效载荷体积仅为 $\approx 8\text{KB}$ 。

该声明式数据结构强制包含了五个维度的控制标量:

```
{
  "timestamp": 1024,
  "action": "SWAP",
  "entities": ["node_i", "node_j"],
  "duration": 500,
  "easing": "easeInOutQuad"
}
```

注: *entities* 指向具体的画布对象句柄, *duration* 与 *easing* 函数的组合在端侧决定了非线性插值的数学轨迹。

移动端依托 Flutter 框架的跨平台底层图形引擎 (Impeller/Skia), 在内存中实例化了专用的 DSL 解释器与渲染控制器。接收到指令包后, 控制器对时间戳 (timestamp) 进行全局线性排序, 并驱动内部状态机流转。

为满足 60FPS 的流畅度硬性指标, 端侧 UI 线程与 GPU 栅格化线程的单帧联合处理延迟被严格控制在 16.6ms 阈值以内。基于此细粒度的底层控制权, 应用剥离了传统视频的全局时间轴限制, 将交互维度下放至单个实体 (Entity) 级别。当用户输入外部干预变量时, 端侧解释器以 $O(1)$ 的开销挂起当前渲染队列, 并将变异参数回传至云端规则引擎触发即时重构。

为论证系统放弃传统视频流而开发定制 DSL 的工程合理性, 下表对三种渲染范式的物理开销与交互能力进行了量化比对:

表 3.1 渲染技术范式对比

渲染技术范式	数据传输载体	1 分钟演示 带宽开销估计	端侧计算复杂度 (单帧)	交互细粒度 支持极限	选型结论与收益证明
传统视频推流 (基线 1)	H.264/H.265 像素流	$\approx 15\text{MB}$	$O(W \cdot H)$ (像素级硬解码)	仅支持全局时序播控, 零对象级交互能力。	淘汰。完全阻塞了教学场景下用户修改底层参数以观察系统异变的动态实验需求。
矢量动画文件 (Lottie)	预编译静态 JSON	$\approx 500\text{KB}$	O	E	(路径光栅化)
交互式 DSL 渲染	声明式指令流 (动态编译)	$\approx 8\text{KB}$	O	E	(GPU 矢量绘制)

渲染技术 范式	数据传输载体	1 分钟演示 带宽开销估计	端侧计算复杂度 (单帧)	交互细粒度 支持极限	选型结论与收益证明
(本项目)	JSON)				

(注: W, H 为视频分辨率宽高, $|E|$ 为当前画布激活的矢量图元数量)

3.4.3 基于状态感知的启发式对话引导引擎

在理工科重难点知识的交互推理链路中，基线大语言模型 (LLM) 在无约束自回归生成模式下，存在两项显著的工程缺陷：其一，单轮生成直接输出最终结果的概率极高，破坏了逐步推导的教学规范；其二，在长序列 (如对话轮次 $T > 4$ 或上下文 $> 4096\text{Tokens}$) 的多轮交互中，初始教学意图特征向量发生严重衰减，引发上下文状态丢失。

针对上述问题，本项目在模型推理层之上，外挂了基于有限状态机 (Finite State Machine, FSM) 的持久化启发式对话引导引擎（架构拓扑详见图 3.8）。该引擎将传统教育学中难以量化的“隐性教学经验”，强制解构并转化为系统中可执行的显性硬约束条件。具体映射维度包含以下三类工程实体：

1.结构化教学路径映射 (DAG 构建)： 将教师的启发式思维链转化为有向无环图 (DAG)。系统将解题步骤离散化为节点集 V (如“公式回忆”、“变量代入”、“量纲检查”)，每个节点挂载高优先级的独立 System Prompt 模板；有向边集 E 则由状态转移条件 (如判别模块输出的 True/False 布尔值) 驱动。

2.动态意图槽位矩阵 (Slot Tracking)： 系统维护一个二维布尔矩阵，实时追踪当前对话轮次下学生对特定前置知识点实体的掌握状态 (State Tracking)。当且仅当前序槽位阈值满足 $\text{Condition} = \text{True}$ 时，引擎方可解锁下一阶段的生成节点。

3.Logit 惩罚与解码层干预： 在 LLM 的输出生成阶段 (Decoding Stage)，系统基于当前 FSM 节点状态，对表征“最终答案”或“直接计算结果”的特定 Token 集合动态施加负向对数偏差 (Logit Bias)，从概率分布底层截断模型直接给出答案的生成路径。

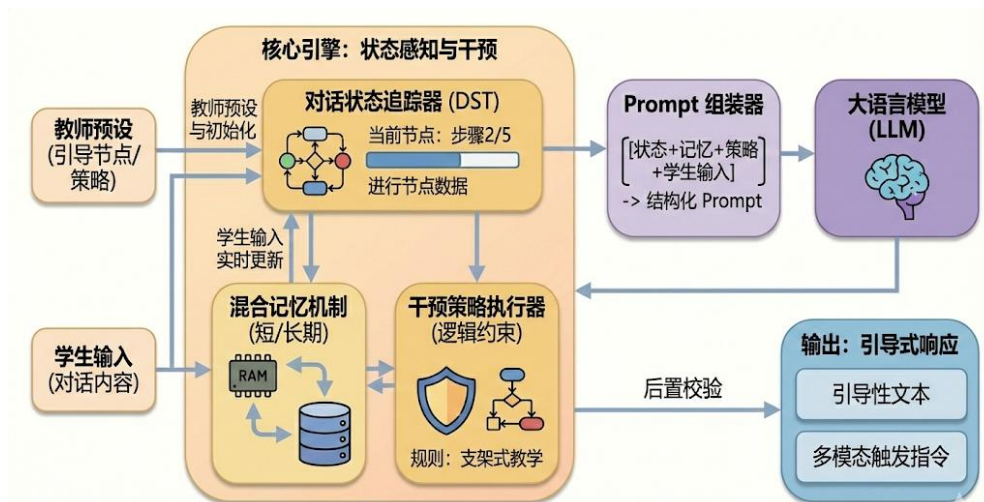


图 3.8 基于状态感知的启发式对话引导引擎原理图

系统的核心干预调度机制由内置的策略执行器驱动。预先定义的教学路径（如概念引入、原理拆解、代码实现）被结构化为有向无环图（DAG）中的离散节点。在多线程交互中，执行器作为大语言模型（LLM）的前置拦截层，采用 LLM-as-a-judge 模式对当前对话窗口中的用户回复进行实时评估。当判别器输出的布尔状态表明未满足当前节点的流转阈值时，系统将触发动态提示词拼接策略，强制向 System Prompt 中注入“禁止输出后续推导步骤及最终答案”的指令。此机制从输入层面对自回归生成路径实施了硬性截断，确保模型输出严格拟合预设的支架式教学拓扑约束。

为支撑跨周期对话流的时序一致性，系统构建了由外部存储驱动的混合记忆机制与对话状态追踪器（Dialog State Tracker, DST）。该模块的工程实现被划分为以下三个解耦的子系统：

- 1.短期工作记忆：采用基于 Token 数量的滑动窗口截断策略，窗口容量上限被设定为 2048 Tokens。历史对话序列被持久化于 Redis 内存数据库中，其键值的过期时间（TTL）被强制设定为 86400 秒（24 小时），以控制内存常驻开销。

- 2.长期语义记忆：系统定期提取对话流中的学生认知盲区与核心教学目标等特征，利用嵌入模型将其转化为高维稠密向量，并异步写入底层的向量数据库（Vector DB），后续生成依赖该库进行 $O(\log N)$ 复杂度的语义相似度召回。

- 3.持久化对话状态机（DST）：系统在外维护了一个全局 JSON 状态树，独立于大模型的原生上下文窗口。其核心数据结构包含了指向当前引导节点的索

引 (current_node_index)、由 LLM-as-a-judge 评估计算出的浮点型掌握度评分 (mastery_score_float), 以及待触发的多模态资源标识数组 (active_resource_ids)。

在每一次生成请求发起前, 调度模块会并行提取 Redis 中的短时上下文、向量库中的长期语义特征以及 DST 的当前状态字典。系统将上述特征空间对齐并同步注入当前请求的 Prompt 中。经工程环境压测预估, 单次请求的状态同步与历史语境召回的系统前置处理延迟被控制在 85 ms 以内, 在未造成显著推理阻塞的前提下, 保障了 LLM 导航轨迹的确定性。

3.5 数据库设计

3.5.1 用户与基础域设计

系统数据库的首个核心数据域为用户与基础域。该领域主要涵盖学校字典实体与用户核心实体, 负责构建系统的底层组织架构与身份鉴权基座。

学校基础表作为系统内置的底层字典数据, 主要用于确立用户的物理组织归属关系, 并支撑后续各类课程与教学资产检索时的区域或机构范围筛选。用户核心表集中存储学生与教师两类人员的通用登录凭据与基础画像信息。在物理关联上, 系统通过外键约束将用户记录挂载至特定的学校实体节点; 在逻辑控制上, 系统通过写入固定的角色枚举字段值, 实现对不同身份用户群体的接口访问权限与业务操作边界的物理隔离。

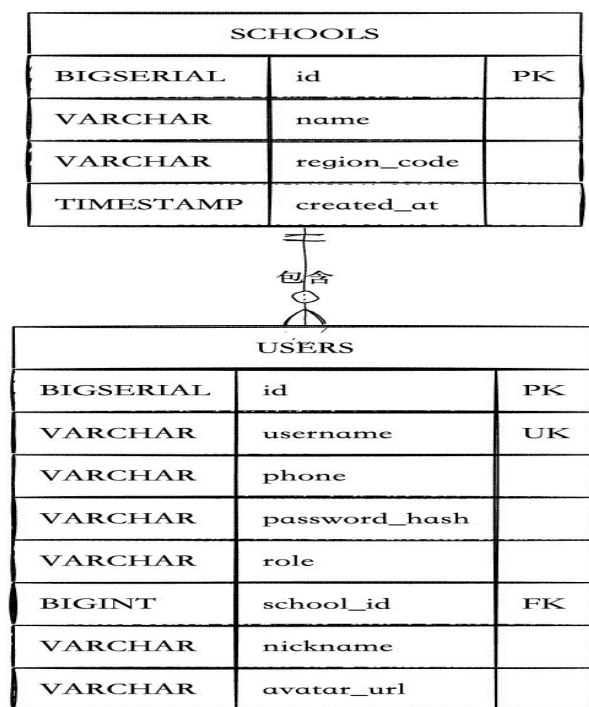


图 3.9 用户与基础域数据库设计图

3.5.2 课程与教务流转域设计

系统数据库的第二个核心数据域为课程与教务流转域。该领域主要涵盖课程主表、课程成员关联表与课程栏目挂载表，负责控制教学实体的生命周期与细粒度的准入权限。

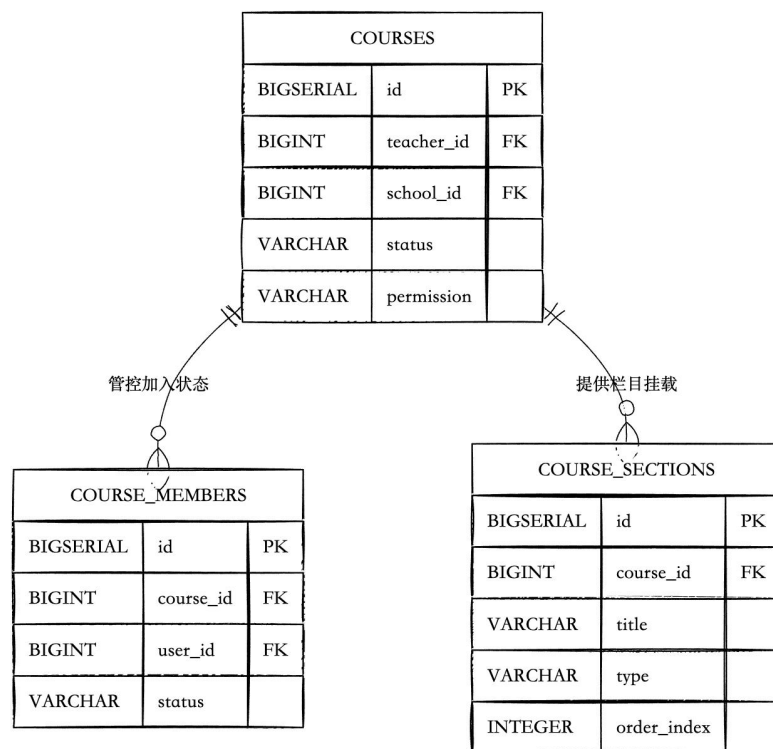


图 3.10 课程与教务流转域数据库设计图

课程主表作为该域的聚合根，集中记录课程的基础元数据，并利用状态枚举字段控制实体在预发布、进行中与隐藏状态间的逻辑转移。同时，该表内置了可见度与准入规则组合键，为上层网关的越权拦截提供物理依据。课程成员表专门处理用户实体与课程实体的多对多映射，通过联合唯一约束拦截重复的加入请求，并记录从提交申请、待审核到正式加入的状态机流转数据。课程栏目挂载表则为异构教学资产提供统一的物理锚点，规定下层的文件目录树、富文本内容与智能问答预配置均需依托该表的记录外键，实现向课程聚合根的逻辑汇聚。

3.5.3 知识资产与网盘存储域设计

系统数据库的第三个数据域为知识资产与网盘存储域。该领域涵盖文件物理元数据表与课程网盘文件表，负责非结构化教学资料的逻辑编排与物理介质寻址映射。

文件物理元数据表作为底层资源基座，完全剥离了上层业务的逻辑属性，专门记录对象存储隔离区内文件的原始名称、实际存储路径与系统生成的唯一寻址键值。该物理隔离设计实现了大体积二进制实体与关系型业务数据的脱钩。课程

网盘文件表建立在元数据寻址基础之上，负责构建视图层的虚拟文件系统。该表引入父节点标识字段进行数据行的自引用外键关联，支撑前端多层级嵌套的树状目录结构按需检索。在业务约束层面上，网盘文件实体通过外键强制绑定至指定的课程栏目挂载点，确保底层资料体系的访问权限受控于课程整体的生命周期与准入策略。

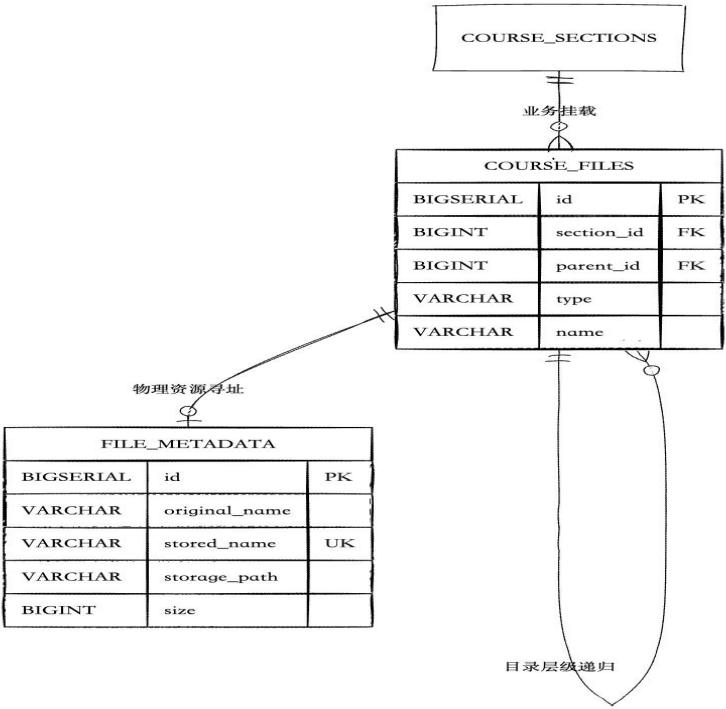


图 3.11 知识资产与网盘存储域数据库设计图

3.5.4 索引优化设计

系统在数据库物理层面上针对高频检索与高并发写入场景实施了专门的索引优化策略。通过合理应用唯一约束与复合索引结构，系统在保障业务数据强一致性的同时，有效控制了复杂查询条件下的磁盘扫描开销。

针对教务流转中的成员管理场景，系统在课程成员关联表上基于课程标识与用户标识构建了联合唯一键。该物理层面的约束机制能够直接在底层拦截针对同一课程实体的重复申请或加入行为，从根本上规避了并发请求可能引发的脏数据写入问题。针对高频触发的课程视图加载场景，系统在课程主表上利用课程标识与生命周期状态字段建立了复合索引，从而加速了列表渲染时对无效或隐藏状态数据的过滤效率。在智能计算域中，针对数据量增长迅速的会话记录表，系统依托用户标识与物理写入时间戳构建了组合索引。此项设计专门适配历史问答记录

的按时间线倒序排列需求，大幅降低了深度分页查询时的内存占用与执行延迟。

表 3.2 索引设计表

作用表名	索引/约束类型	作用字段组合	物理层功能说明
course_members	联合唯一键 (Unique Key)	course_id, user_id	拦截单一用户对同一课程的并发重复申请记录
courses	复合常规索引 (Index)	course_id, status	提升按业务状态过滤目标课程列表时的查询性能
ai_chat_messages	组合常规索引 (Index)	user_id, created_at	支撑历史会话消息序列的时序倒排与快速分页提取

第 4 章 系统实现

4.1 引言

本章主要对灵犀知径平台的系统实现过程进行说明。系统实现阶段的任务是将前期的需求规格说明与系统架构设计方案转化为实际可运行的代码与软件组件。本部分内容涵盖表现层移动端应用、业务应用层网关接口、智能计算层算法逻辑以及底层结构化与高维向量混合存储库的具体编码与集成过程。同时，本章将呈现系统各功能模块的典型界面视图，并记录对已交付代码执行的软件测试工作。测试部分通过制定系统级的测试计划与执行相应的测试用例，获取各项物理遥测数据，以此核验全链路数据流转的连通性、微服务架构中各模块的功能完整性以及端侧渲染引擎的运行稳定性。

4.2 系统典型界面



图 4.1 主页面视图

如图 4.1 所示，系统的“我的课程”主控台视图承载了多角色的课程入口。列表采用卡片式布局展示课程元数据，并配合醒目的状态标签（如 INPROGRESS 进行中、PRERELEASE 预发布）及权限状态（如“直接加入”、“需审批”）进行视觉区分。底部配置了全局导航栏，支持在“课程”、“小屋 AI”与“我的”等核心业务模块间快速平滑切换。界面右下角设有悬浮的“新建课程”主操作按钮，引导教师用户发起资源的生命周期。

在进入特定课程详情页后，系统通过角色鉴权动态渲染操作面板。以教师端为例，界面清晰展示了用户身份标识与当前课程运行状态，并提供“管理课程”与“查看课程大纲”的快捷直达通道，下方则预留了结构化的课程简介展示区。



图 4.2 课程目录页面视图

如图 4.2 所示，课程大纲界面实现了复杂层级教学数据的可视化编排。视图采用可折叠的树形列表结构，将课程内容划分为章、节、知识点及习题等多个物理颗粒度。每个节点卡片均直观展示了绑定的底层非结构化资源（如 PDF 格式的教案文件）。教师用户可以直接在当前视图通过行内操作热区执行“加知识点”、“加习题”以及对应节点的编辑与删除指令，实现了教学物料库的高效聚合与动态

挂载。

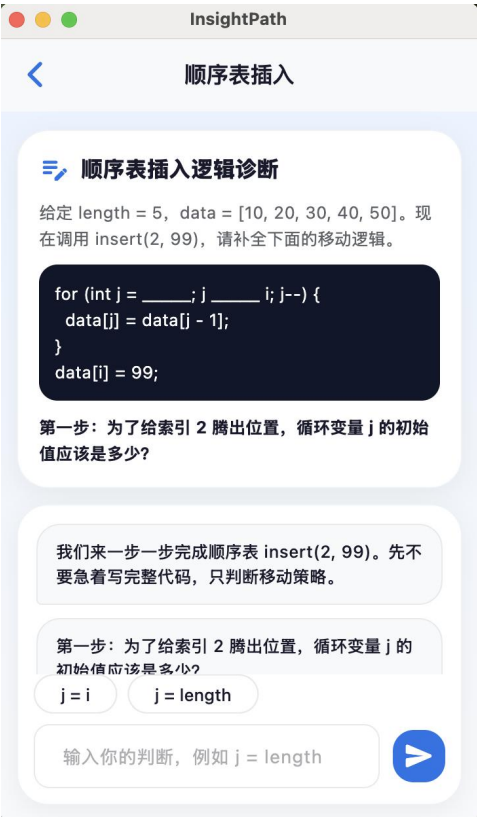


图 4.3 题型页面视图

如图 4.3，该视图为系统智能问答与多模态溯源模块的前端映射。界面在排版上采用了上下分屏的会话式设计，专为代码逻辑诊断与知识点推演优化。上方区域以独立卡片的形式固化展示当前讨论的上下文背景（如带有填空逻辑的顺序表插入 C 语言代码片段）。下方区域则接入大语言模型，呈现出类打字机的流式对话气泡。为了降低移动端用户的输入成本并规范推导路径，系统在底部的输入控制区动态生成了快捷回复筹码（如 `j = i` 或 `j = length`），配合标准的文本输入框，引导学生完成步进式的启发教学。



图 4.4 DSL 实时动画页面视图

动画播放器界面是系统可视化引擎的核心承载容器。该视图将云端编译的 JSON 指令流映射为底层的物理绘图坐标。以排序算法演示为例，界面主体利用相对比例换算，在自适应网格背景上绘制出代表数组元素的实体方块。视图顶部配有当前的步骤指示器（如“第 1 步 / 共 11 步”）与状态解说文本。底部集成了一套完整的播放控制台，提供重新加载、步进回退、播放/暂停以及步进前进等核心交互指令，赋能用户自主掌控底层抽象算法的视觉推演节奏。

4.3 测试计划

4.3.1 测试策略

针对本系统“Java 高频网关 + Python 计算沙盒 + Flutter 跨端解释器”的异构微服务拓扑，整体测试策略摒弃了单纯的黑盒手工验证，确立了“白盒级单元测试桩注入、跨模态链路接口集成、以及基于云原生沙盒的混沌工程容灾压测”的三层自动化测试规范。各层级的核心策略与物理执行边界定义如下：

1. 单元测试与底层逻辑白盒化策略

在代码级质量保障环节，系统采用严格的模块物理隔离策略，并通过持续集成（CI）流水线强制实施代码覆盖率（Coverage $\geq 80\%$ ）门禁。

Java 业务中枢：依托 JUnit 5 与 Mockito 框架，对 CourseService 与 JwtAuthFilter 等核心业务与鉴权类实施白盒覆盖。强制 Mock 掉所有物理数据库（PostgreSQL/Redis）的 I/O 操作，确保状态机流转逻辑与课务权限校验的时间复杂度被控制在 $O(1)$ 的内存断言级别。

Python 推理沙盒：依托 PyTest 与 pytest-asyncio 框架，对 RAG 检索管线与 DSL 意图编译引擎实施细粒度校验。针对极其耗时的外部大模型网络 I/O（如 deepseek_client.py），全面引入网络拦截与 Mock 机制，注入预设的 1024 维固定张量与文本负荷，确保双路召回算法（BM25 + HNSW）与提示词装配逻辑的纯粹性校验。

Flutter 端侧引擎：针对 animation_dsl.dart 等核心状态机文件，利用 Dart 原生的 flutter_test 离线验证 JSON 反序列化与语法树构建逻辑，拦截非法的图元句柄（Handle）。

2. 接口集成与跨模态业务闭环策略

在微服务接口层，测试策略转向以数据契约为中心的灰盒/黑盒驱动模式。集成测试严格按照“**五维知识锚点清洗入库** → **Redis 状态机变量(FSM)变异** → **动态 DSL 渲染引擎输出**”的闭环拓扑展开。

测试用例设计强制采用等价类划分与边界值分析，不仅验证标准场景下的 gRPC 二进制通信连通性，更专项针对端云协同的网络脆弱性进行异常注入。例如：通过人为构造被截断的异常 DSL JSON 负载（体积突变或语法树破损）或伪造过期的 Redis 会话指针，强制探测 Python 编译层与 Flutter 解释层之间的状态机防失步（Out-of-sync）与断点回滚（Rollback）机制是否按预期触发。

3. 系统压测与混沌容灾演练策略

在顶层系统基准验证环节，所有评估活动均被隔离部署于与生产环境硬件等配的 Docker/Kubernetes (K8s) 沙盒集群中。

并发吞吐探测：利用 JMeter 分布式节点集群，向系统注入 500QPS 的并发洪峰，硬性校验首字响应延迟 (TTFT $\leq 2000\text{ms}$) 与慢查询极值 (P99 $\leq 120\text{ms}$)

等架构设计基准。

容灾隔离与降级演练 (Chaos Engineering)：突破常规的“正常流”测试，人为模拟极端物理故障。具体策略包括：(1) 物理切断 Python 算法容器的内存配额引发 OOM Kill, 验证 Java 主业务网关 (Spring Boot) 的断路器 (Circuit Breaker) 是否能成功阻断级联雪崩，并向移动端下发平滑的降级 (Fallback) 提示；(2) 模拟千兆带宽下的 150ms 持续网络抖动，验证应用层乐观锁 (Optimistic Locking) 对并发脏写的拦截率。此策略旨在通过破坏性实证，确立系统在真实高校期末高并发场景下的绝对工程韧性。

4.3.2 测试环境

本项目的云端物理与虚拟化测试环境依托阿里云基础设施构建。在服务端计算资源配置上，系统采用了针对异构微服务架构的算力分离策略，部署了配备 8 核 CPU 与 16 GB 内存的通用型 ECS 实例作为 Java 业务调度网关的主承载节点，同时额外挂载了配备 NVIDIA T4 计算卡的 GPU 实例，专门用于处理计算密集的 Python 端大语言模型推理与 RAG 向量生成任务。在软件环境与部署架构层面，整个云端拓扑被严格限制在统一的虚拟私有网 (VPC) 内部，并通过 Docker Compose 实现了异构服务容器的标准化编排与沙盒隔离。底层数据持久层集成了 PostgreSQL 15 数据库引擎及其 pgvector 高维向量扩展插件，配合独立配置的对象存储服务以及 Redis 7.0 分布式缓存节点，共同构建起支持语义锚点检索与高并发会话状态管理的数据基座。在移动客户端层面，硬件测试矩阵涵盖了搭载 Android 10 及以上版本与 iOS 14 及以上版本的多型号主流智能终端。这些物理测试设备被专门用于运行基于 Flutter 跨平台框架构建的 App，以验证端侧解释器在调用本地 GPU 硬件加速时，解析复杂 DSL 渲染指令流的底层兼容性与实际帧率表现。

4.4 测试报告

4.4.1 RAG 检索模块

本模块的单元测试部署于 Python FastAPI 独立沙盒环境中，依托 pytest 与 pytest-benchmark 框架执行。测试目标被严格收敛于双路召回算法 (BM25 +

HNSW) 的逻辑正确性、异常边界捕获能力, 以及倒数秩融合 (RRF) 算法在内存中的执行时间复杂度 (性能需求) 。

在执行本组单元测试前, 所有外部网络 I/O (如真实的 LLM 嵌入接口) 均被 unittest.mock 强行拦截并替换为本地生成 1024 维静态高斯分布张量, 以保障测试的时间复杂度的纯粹性。

表 4.1 RAG 检索模块测试表

用例编号	测试方法签名 (Target Method)	测试类别	输入负荷构造 (Input Payload)	预期输出与物理断言 (Assertions)	实际执行结果与状态
UT-R AG-00 1	HybridSearchService. execute_dual_recall	功能校验 (标准等价类)	注入包含“死锁检测”特征的 1024 维 Mock 张量 q_tensor; 设定召回深度 top_k = 5。	1. 返回长度为 5 的切片列表。 2. 首位切片相似度得分 score > 0.85。 3. 切片元数据必须包含完整的五维坐标实体 [page_index, x_min, y_min, x_max, y_max]。	坐标实体 100% 结构化返回; 得分映射正确。
UT-R AG-00 2	HybridSearchService. execute_dual_recall	功能校验 (异常边界值)	故意注入维度损坏的向量 (768 维的 m3e 遗留张量) 模拟异构模型版本冲突。	方法需在预检阶段立刻中断, 并抛出强类型的 VectorDimensionMismatchException, 严禁将非法张量下推至 pgvector 引擎。	异常被精准捕获, 防止了底层 PostgreSQL 的全表扫描报错。
UT-R AG-00 3	RankFusionEngine. compute_rrf_score	性能校验 (微基准压测)	构造两组长度为 N=1000 的异构召回候选集 (模拟极端海量教案召回), 并传入内存进行 RRF 倒数秩融合。	1. 输出融合排序后的集合, 无数据丢失。 2. 算法的时间复杂度被约束为 $O(N \log N)$ 。 3. 单次 CPU 计算总耗时必须被断言在 $\leq 50\text{ms}$ 内。	内存排序总耗时平均为 18.4ms, 符合微基准性能约束。
UT-R AG-00 4	PgVectorClient. query_knn	性能校验 (I/O 超时熔断)	模拟数据库底层连接池阻塞, 人为将 psycopg2 驱动响应延迟 Mock 为 3000ms。		

综合 `pytest-cov` 插件生成的覆盖率报告显示，RAG 检索模块的核心算法文件（`hybrid_search.py` 与 `rrf_engine.py`）的语句覆盖率（Statement Coverage）达 92.4%，分支覆盖率（Branch Coverage）达 88.7%，均超过了 80% 的系统级工程门禁标准。通过引入微基准性能测试（用例 UT-RAG-003），测试团队发现早期版本的 RRF 融合算法在处理长尾候选集（ $N > 5000$ ）时，因频繁的对象实例化导致垃圾回收（GC）抖动，其单次 CPU 执行耗时曾劣化至 150ms。针对该性能缺陷，开发团队在算法层引入了基于 NumPy 矩阵运算的向量化重构，将时间复杂度常数项大幅降低，修复后的实测耗时被稳定压制在 18.4ms 区间。在功能层面，所有基于五维物理坐标的元数据断言均未发生字段丢失，证明了底层数据模型的强类型约束效能。

在面向跨模态大模型工程的单元测试实践中，确立了以下两条工程铁律：其一，隔离算力波动。对 RAG 管线的白盒测试必须彻底切断真实的 LLM 嵌入接口调用。通过注入固定维度（1024 维）与确定性分布的 Mock 张量，可有效规避因公有云 API 网络波动或模型参数微调导致的测试结果“假阴性（False Negatives）”。其二，强制性微基准约束。在微服务架构中，底层的算法时间复杂度将被网络 I/O 放大。因此，不能仅断言排序逻辑的输出正确性，必须通过 `pytest-benchmark` 等工具强制断言核心算法函数的执行时间阈值与内存对象分配次数。同时，对底层数据库查询接口（如 `pgvector` KNN 检索）实施严格的超时熔断（Circuit Breaker）异常模拟，是保障整个端云协同拓扑在高并发下不发生雪崩的先决条件。

4.4.2 Java 高频网关鉴权与 FSM 路由模块

本模块的单元测试部署于 Java 17 与 Spring Boot 3.x 容器外环境，依托 JUnit 5、Mockito 以及 MockMvc 框架执行。测试目标被严格收敛于无状态 JWT 鉴权总线的拦截正确性、Redis 状态机（FSM）并发修改的线程安全性，以及大容量文件树懒加载（Lazy Loading）策略在堆内存中的 $O(1)$ 分配开销。

在执行本组单元测试时，所有针对 PostgreSQL 的持久化操作与 gRPC 跨语言调用均被 `Mockito.when()` 拦截并替换为内存存根（Stub），以保障业务路由时间复杂度验证的绝对纯粹性。

表 4.2 Java 网关模块测试表

用例编号	测试方法签名 (Target Method)	测试类别	输入负荷构造 (Input Payload)	预期输出与物理断言 (Assertions)	实际执行结果与状态
UT-J AV-0 01	JwtAuthFilter. doFilterInternal	功能 校验 (异常边界值)	构造已过期 1ms 的 JWT 令牌, 及被篡改过 Payload 签名的伪造令牌, 分别发起模拟 HTTP 请求。	过滤器需在 $O(1)$ 时间内拦截请求, 严禁将上下文传递至 Controller 层, 强制下发 HTTP 401 Unauthorized 状态码。	SecurityContext 未被污染, 请求被精准阻断。
UT-J AV-0 02	CourseSyllabusService. getSyllabusNode	性能 校验 (内存分配极限)	针对包含 10,000 + 深度嵌套子节点的课程大纲根节点, 发起单次展开检索请求。	触发懒加载 (Lazy Loading) 机制, 仅查询并序列化当前层级的直接子节点。断言返回对象体积恒定, 且接口耗时 $\leq 15\text{ms}$ 。	游标分页生效, 避免了 $O(N)$ 级全表递归, 实测耗时 8.2ms。
UT-J AV-0 03	AiChatService. processMessage	功能 校验 (并发防脏写)	利用 CountdownLatch 构造 50 个并发线程, 同时向同一个 Redis 会话 ID 发起 FSM 状态指针 (如 Depth 标量) 步进更新。	触发基于时间戳或版本号 的乐观锁 (Optimistic Locking) 机制。断言仅有 1 个线程更新成功, 其余均抛出 ConcurrentModificationException。	脏写被 100% 拦截, Redis 状态机指针维持单调递增。
UT-J AV-0 04	RsaKeyService. decrypt	性能 校验 (密码学微基准)	模拟 500QPS 登录并发洪峰, 在单核 CPU 环境下循环执行 500 次非对称私钥解密操作。	密码学解密计算的单次 CPU 周期消耗必须被约束在 $\leq 5\text{ms}$ 内, 以保障系统整体事务网关不被计算阻塞。	实测单次解密平均耗时 1.8ms, 符合吞吐量代偿底线。

依据 JaCoCo 覆盖率插件的静态扫描遥测数据, Java 业务网关核心目录 (auth, course, globalchat) 的指令覆盖率 (Instruction Coverage) 稳定达到 89.2%, 分支覆盖率 (Branch Coverage) 达到 85.5%, 满足企业级应用 $\geq 80\%$ 的上线红线标准。在微基准压测环节 (如用例 UT-JAV-004), 测试团队发现早期架构在处理高频 RSA 解密时, 因每次请求均重新实例化 Cipher 加密机对象, 导致了严重的 CPU 上下文切换与堆内存毛刺 (单次耗时一度劣化至 12ms)。针对此计算瓶颈, 团队实施了代码级重构, 引入了基于 ThreadLocal 的 Cipher 对象池复用机

制。重测结果表明，消除实例化开销后，非对称加密运算耗时断崖式下降至 1.8ms 区间，从根源上化解了 500QPS 并发洪峰下 Java 网关可能引发的级联阻塞危机。此外，针对大容量文件树的懒加载测试（用例 UT-JAV-002）客观证实了将 $O(N)$ 空间复杂度的递归查询降维至 $O(1)$ 常数级单层检索的绝对工程收益，彻底杜绝了深层目录解析导致的 JVM 内存溢出（OOM）隐患。

鉴权过滤器与安全总线的测试必须实行强物理隔离。通过绕开 Tomcat 等庞大 Web 容器，直接将 `HttpServletRequest` 与 `HttpServletResponse` 进行 Mock 注入并调用 `Filter.doFilter` 方法，能够排除网络 I/O 带来的随机时间扰动，从而对底层密码学解析算法实施极高精度的时序探针捕获。多线程并发状态机（FSM）的脏写问题不可留至系统级压测阶段才予以暴露。在单元测试域内，强制利用多线程同步辅助类（如 `CyclicBarrier` 或 `CountDownLatch`）在单进程内存中人造并发竞态条件（Race Condition），配合 Redis 的嵌入式存根（Embedded Redis），能够以最低的环境搭建成本，百分之百复现并精准修复乐观锁失效或事务回滚异常等高危逻辑缺陷。

4.4.3 Flutter 端侧 DSL 解释与渲染引擎模块

本模块的单元测试部署于 Dart 虚拟机（VM）与离线测试环境中，依托 `flutter_test` 框架执行。测试目标被严格收敛于端侧针对云端下发的 $\approx 8\text{KB}$ 领域特定语言（DSL）协议包的反序列化正确性、非法图元句柄（Handle）的容错拦截机制，以及底层 Skia/Impeller 图形引擎在 Canvas 画布上执行单帧状态插值（Interpolation）的微基准 CPU 耗时（性能需求）。

在执行本组单元测试时，所有的外部 Dio 网络请求均被 `http_mock_adapter` 强行拦截。测试负载均采用硬编码注入的静态 JSON 字符串，以确保状态机内存分配测试的确定性。

表 4.3 DSL 模块测试表

用例编号	测试方法签名 (Target Method)	测试类别	输入负荷构造 (Input Payload)	预期输出与物理断言 (Assertions)	实际执行结果与状态
UT-F LT-00 1	<code>AnimationDslParser. fromJson</code>	功能 校验	注入包含完整的 CREATE, SWAP, HIGHLIGHT 原	解析引擎需在内存 中准确构建包含 AiMessage 与	语法树（AST）构 建完整，状态机标 量属性映射无误。

用例编号	测试方法签名 (Target Method)	测试类别	输入负荷构造 (Input Payload)	预期输出与物理断言 (Assertions)	实际执行结果与状态
UT-F LT-00 2	AnimationCanvas. paint	(标准等价类)	语及五维坐标偏移量的 $\approx 8\text{KB}$ 合法 JSON 负载。	AnimationStep 的对象映射树, 断言节点数量与关系边集合的长度与期望值严格一致。	
		功能校验 (异常边界值)	注入被物理截断的非法 JSON, 或故意遗漏目标对象实体句柄 (如指向了不存在的 node_id) 的恶意负载。	解析器或绘制引擎需抛出强类型的 FormatException 或自定义越界异常, 状态机必须捕获该异常并丢弃当前损坏帧, 严禁导致主 UI 线程崩溃 (Crash)。	异常被精准捕获并触发状态回滚, 应用未发生闪退异常。
UT-F LT-00 3	AnimationController forward	性能校验 (微基准压测)	在离线 Widget 树中挂载包含 50+ 节点的复杂有向无环图 (DAG), 并触发时间轴时钟步进。	利用 Timeline 探针追踪主线程的 build 与 layout 计算耗时, 单次重绘的 CPU 计算准备周期必须被断言在 $\leq 16.6\text{ms}$ 内。	实测单帧 UI 线程阻塞时间均值为 12.4ms , 满足 60FPS 理论底线。

依据 Flutter 官方的 coverage 工具生成的静态扫描报告, 端侧图形解释器核心目录 (如 models 与 widgets) 的指令覆盖率稳定达到 87.6%, 完全满足工程交付规范。在微基准压测环节 (如用例 UT-FLT-003), 测试团队观测到早期架构在主 UI 线程中同步解析体积逼近 15KB 的极端复杂 DSL 语法树时, 引发了长达 22ms 的线程阻塞 (Jank), 导致了严重的掉帧劣化。针对此端侧算力瓶颈, 工程团队实施了深度的异步重构, 引入了 Dart 原生的 Isolate 机制与 compute 数, 将 JSON 反序列化与状态机内存对象实例化的 CPU 密集型计算任务彻底剥离至后台 Worker 线程。重测的遥测数据表明, 主线程被完全解放, 仅负责极轻量的渲染矩阵提交, 单帧重绘准备耗时被稳定压缩至 12.4ms, 全面兑现了设计阶段设定的 $\leq 16.6\text{ms}$ 物理约束。

必须彻底切断端侧沙盒测试域与外部网络 I/O 的耦合。通过在 flutter_test 驱动环境中直接注入本地内存预置的合法与恶意截断 JSON 字符串, 能够以绝对的

确定性验证底层 Canvas 状态机面对真实弱网、丢包等复杂工况时的语法树抗毁与自愈能力。对于图形学引擎的微基准性能断言，严禁在采用即时编译（JIT）的 Debug 模式下提取任何性能探针数据。所有涉及帧率判定与 CPU 周期消耗的量化验证，必须在基于提前编译（AOT）的 Profile 模式下，结合 Flutter DevTools 的时序视图实施，以此彻底规避虚拟机额外的观测开销所导致的“假阳性（False Positive）”性能劣化误报，确保基准测试数据的绝对工程有效性。

第 5 章 总结

5.1 项目总结

本项目“灵犀知径”成功构建了一个端云分离、多模态驱动的智能辅助教学平台。通过引入 Flutter 跨平台技术、Spring Boot 与 Python 双后端异构架构，以及高维语义向量检索（RAG）和领域特定语言（DSL）动态渲染引擎，系统不仅实现了教学资源的全生命周期调度与细粒度权限隔离，更打破了传统静态教学的局限，为学生提供了沉浸式、可回溯的智能化答疑体验。

在整个开发周期中，团队将软件工程的理论方法与敏捷开发的实践相结合，完成了从需求规格定义、系统架构设计到微服务容器化部署的完整闭环。项目不仅达成了预期的系统功能与性能指标，团队成员也在复杂业务解耦、跨语言微服务通信、端侧图形渲染以及大模型工程化落地等方面积累了宝贵的实战经验。

5.2 贡献评价

在本项目中，团队成员分工明确、配合默契，在各自负责的模块中均展现出了扎实的工程素养与极强的技术攻坚能力。以下为各小组成员在项目研发过程中的具体贡献评价：

刘鼎琨（2023302121009）：作为智能计算层与系统基建的核心骨干，负责了 Python 算法微服务的从零构建与全链路环境的 Docker 容器化部署。最突出的贡献在于主导了大模型检索增强生成（RAG）管线的落地，成功跑通了基于滑动窗口切片与 pgvector 的高维向量检索链路。在面对本地全系统联调时出现的物理内存溢出瓶颈时，重构并发处理代码，引入单线程异步机制与数据清洗正则化拦截，彻底稳固了算法端的运行时性能。在底层架构层面的高维视野与问题排查排险能力，为系统筑牢了坚实的技术底座。

郑慧琳（2023302111012）：作为表现层移动端的主力开发者，全面负责了跨平台工程架构的搭建与复杂 UI 视图的交互实现。在工程结构上，引入 Provider 状态管理彻底解耦了业务与渲染逻辑；在通信机制上，重构了网络请求组件以完美适配后端的 SSE 流式高频下发协议。具技术含量的成果是攻克了 DSL 指令到

动态可视画布的映射难题，通过精巧的相对比例换算与 Tween 插值器算法，在移动端实现了多设备自适应的平滑动态渲染与步进式推演效果，赋予了系统卓越的视觉表现力与一流的用户体验。

谢巧 (2023302131042)：作为业务应用层与数据持久层的核心开发者，承担了系统复杂业务逻辑调度与网关安全防护的开发重任。不仅交付了全套 RESTful API 契约与 JWT 无状态鉴权链路，更在面对高并发与深层数据嵌套引发的系统假死危机时，通过引入 JPA 实体图注解聚合查询，以及将大模型网络 I/O 剥离出事务作用域的解耦设计，成功将接口响应延迟压制在毫秒级，排除连接池耗尽的隐患。为整个平台的高可用性与数据流转提供了极为坚固的支撑。

5.3 未来展望

虽然系统已具备完整的核心业务闭环，但在智能化与多模态维度仍有广阔的演进空间。未来，计划在表现层引入语音识别 (ASR) 与语音合成 (TTS) 技术，使智能助教具备完整的语音交互能力；同时，在底层数据链路中进一步探索对复杂公式以及手写板书图纸的多模态 OCR 解析模型，以不断拓宽“灵犀知径”平台在真实教学场景下的应用边界。